

TRAVAUX PRATIQUES

BIG DATA

TP5

Pipeline Big Data End-to-End

Durée : 3h30 M2TL DIGITAL CAMPUS · Prérequis : TP1 à TP4

Objectifs du TP

Ce TP est le point d'orgue des quatre séances précédentes. L'objectif est de construire un pipeline Big Data complet, de l'ingestion des données jusqu'à la consultation des résultats, en mobilisant l'ensemble des technologies vues en cours.

- Apache Kafka — collecte et transport des événements en temps réel
- Apache Spark Structured Streaming — traitement du flux
- Apache HBase — stockage des résultats agrégés
- HDFS — archivage brut des événements pour traitement batch

À l'issue de ce TP, vous serez capables de :

- Concevoir et justifier une architecture Big Data de bout en bout
- Implémenter chaque maillon du pipeline et les faire communiquer
- Identifier les points de défaillance et les décisions d'architecture qui en découlent

Outils et Versions

- Apache Kafka 2.13-3.6.1
- Apache Spark 3.5.0
- Apache HBase 2.5.8
- Apache Hadoop 3.3.6
- Docker — même cluster que les TP précédents
- Java 1.8 · Python 3.x · Maven

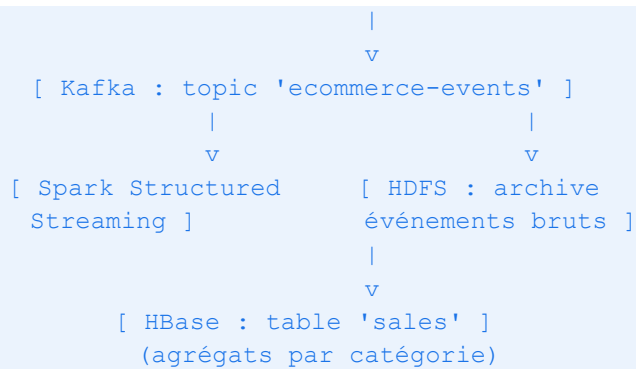
Scénario : Plateforme e-commerce

Vous êtes l'équipe Data Engineering d'une plateforme de e-commerce. Votre DSI vous demande de mettre en place un pipeline capable de :

1. Collecter en temps réel les événements utilisateurs (clics, ajouts au panier, achats)
2. Calculer en continu le chiffre d'affaires par catégorie de produit
3. Stocker ces agrégats dans HBase pour une consultation rapide
4. Archiver tous les événements bruts dans HDFS pour des analyses batch futures

Architecture cible

[Producteur Python]



NOTE

Ce pipeline correspond à la Lambda Architecture : une voie temps réel (speed layer) via Spark + HBase, et une voie d'archivage (batch layer) via HDFS.

Organisation de la séance

Durée	Étape
15 min	Introduction — lecture de l'architecture, répartition des groupes
45 min	Partie 1 — Producteur Kafka (événements e-commerce en JSON)
60 min	Partie 2 — Spark Structured Streaming + écriture HBase
45 min	Partie 3 — Archivage HDFS, vérification du pipeline complet
30 min	Partie 4 — Restitution par groupe et discussion Tech Lead

Démarrage de l'environnement

Démarrer le cluster Docker si ce n'est pas déjà fait :

```

docker start hadoop-master hadoop-worker1 hadoop-worker2
docker exec -it hadoop-master bash
./start-hadoop.sh
./start-kafka-zookeeper.sh
start-hbase.sh
  
```

Vérifier que tous les processus sont actifs :

```

jps
# Attendu (entre autres) :
# NameNode DataNode ResourceManager
  
```

```
# Kafka QuorumPeerMain HMaster HRegionServer
```

Partie 1 — Producteur Kafka

1.1 Création du topic

Créer un topic Kafka dédié à notre flux d'événements e-commerce avec 3 partitions :

```
kafka-topics.sh --create \  
  --topic ecommerce-events \  
  --replication-factor 1 \  
  --partitions 3 \  
  --bootstrap-server localhost:9092  
  
# Vérification  
kafka-topics.sh --list --bootstrap-server localhost:9092
```



QUESTION

Pourquoi choisissons-nous 3 partitions pour ce topic ? Quel impact cela a-t-il sur le parallélisme du consommateur Spark ?

1.2 Producteur Python

Créer le fichier `producer.py` dans le conteneur master pour simuler des événements utilisateurs au format JSON :

```
cat > /root/producer.py << 'EOF'  
import json, random, time  
from kafka import KafkaProducer  
  
producer = KafkaProducer(  
    bootstrap_servers='localhost:9092',  
    value_serializer=lambda v: json.dumps(v).encode('utf-8')  
)  
  
categories = ['Electronique', 'Vetements', 'Maison', 'Livres', 'Sport']  
event_types = ['click', 'add_to_cart', 'purchase']  
  
print('Démarrage du producteur. Ctrl+C pour arrêter.')  
try:  
    while True:  
        event = {  
            'user_id' : f'user_{random.randint(1, 100)}',  
            'event_type' : random.choice(event_types),  
            'category' : random.choice(categories),  
            'amount' : round(random.uniform(5.0, 500.0), 2),  
            'timestamp' : int(time.time() * 1000)  
        }  
        producer.send('ecommerce-events', event)
```

```
        producer.send('ecommerce-events', value=event)
        print(f' → {event}')
        time.sleep(0.5)
except KeyboardInterrupt:
    print('Arrêt du producteur.')
    producer.close()
EOF
```

Installer la bibliothèque et lancer le producteur :

```
pip install kafka-python
python /root/producer.py
```

Dans un autre terminal, vérifier que les messages arrivent bien dans Kafka :

```
kafka-console-consumer.sh \
  --bootstrap-server localhost:9092 \
  --topic ecommerce-events \
  --from-beginning
```



ATTENTION

Laisser le producteur tourner pendant toute la durée du TP. Ouvrir un nouveau terminal pour les étapes suivantes.

Partie 2 — Traitement Spark Structured Streaming

2.1 Préparation de la table HBase

Avant de lancer le job Spark, créer la table de destination dans HBase :

```
hbase shell

# Créer la table avec deux familles de colonnes
create 'sales', 'stats', 'meta'
list
exit
```

Schéma de stockage retenu :

- RowKey — catégorie de produit (ex. Electronique)
- stats:total_amount — chiffre d'affaires cumulé sur la fenêtre
- stats:purchase_count — nombre d'achats
- meta:last_update — timestamp de la dernière mise à jour

2.2 Application Spark — pom.xml

Créer un nouveau projet Maven (groupId : tn.insat.tp5, artifactId : ecommerce-pipeline, version : 1) avec le pom.xml suivant :

```
<dependencies>
  <!-- Spark SQL + Streaming -->
  <dependency>
    <groupId>org.apache.spark</groupId>
    <artifactId>spark-sql_2.12</artifactId>
    <version>3.5.0</version>
    <scope>provided</scope>
  </dependency>
  <!-- Connecteur Spark-Kafka -->
  <dependency>
    <groupId>org.apache.spark</groupId>
    <artifactId>spark-sql-kafka-0-10_2.12</artifactId>
    <version>3.5.0</version>
  </dependency>
  <!-- HBase client -->
  <dependency>
    <groupId>org.apache.hbase</groupId>
    <artifactId>hbase-client</artifactId>
    <version>2.5.8</version>
  </dependency>
  <!-- JSON -->
  <dependency>
    <groupId>com.fasterxml.jackson.core</groupId>
    <artifactId>jackson-databind</artifactId>
    <version>2.15.2</version>
  </dependency>
</dependencies>
```

2.3 Classe principale EcommercePipeline.java

Créer la classe tn.insat.tp5.EcommercePipeline :

```
package tn.insat.tp5;

import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.hbase.HBaseConfiguration;
import org.apache.hadoop.hbase.TableName;
import org.apache.hadoop.hbase.client.*;
import org.apache.hadoop.hbase.util.Bytes;
import org.apache.spark.sql.*;
import org.apache.spark.sql.streaming.StreamingQuery;
import org.apache.spark.sql.types.*;
import static org.apache.spark.sql.functions.*;
import java.util.Iterator;

public class EcommercePipeline {

    static StructType EVENT_SCHEMA = new StructType(new StructField[]{
        new StructField("user_id",      DataTypes.StringType, true, Metadata.empty()),
```

```
new StructField("event_type", DataTypes.StringType, true, Metadata.empty()),
new StructField("category", DataTypes.StringType, true, Metadata.empty()),
new StructField("amount", DataTypes.DoubleType, true, Metadata.empty()),
new StructField("timestamp", DataTypes.LongType, true, Metadata.empty())
});

public static void main(String[] args) throws Exception {

    SparkSession spark = SparkSession.builder()
        .appName("EcommercePipeline").getOrCreate();

    // 1. Lecture du flux Kafka
    Dataset<Row> raw = spark.readStream()
        .format("kafka")
        .option("kafka.bootstrap.servers", "localhost:9092")
        .option("subscribe", "ecommerce-events")
        .load();

    // 2. Désérialisation JSON
    Dataset<Row> events = raw
        .selectExpr("CAST(value AS STRING) as json_str")
        .select(from_json(col("json_str"), EVENT_SCHEMA).as("data"))
        .select("data.*");

    // 3. Filtrer uniquement les achats
    Dataset<Row> purchases = events
        .filter(col("event_type").equalTo("purchase"));

    // 4. Agrégation par catégorie sur fenêtre de 1 minute
    Dataset<Row> aggregated = purchases
        .withColumn("event_ts",
            to_timestamp(col("timestamp").divide(1000)))
        .withWatermark("event_ts", "2 minutes")
        .groupBy(window(col("event_ts"), "1 minute"), col("category"))
        .agg(
            sum("amount").as("total_amount"),
            count("*").as("purchase_count"));

    // 5. Écriture dans HBase via foreachBatch
    StreamingQuery query = aggregated.writeStream()
        .outputMode("update")
        .foreachBatch((batch, batchId) -> {
            batch.foreachPartition((Iterator<Row> rows) -> {
                Configuration conf = HBaseConfiguration.create();
                try (Connection conn = ConnectionFactory.createConnection(conf);
                    Table table = conn.getTable(TableName.valueOf("sales"))) {
                    while (rows.hasNext()) {
                        Row row = rows.next();
                        String cat = row.getString(row.fieldIndex("category"));
                        double total = row.getDouble(row.fieldIndex("total_amount"));
                        long cnt = row.getLong(row.fieldIndex("purchase_count"));
                        Put put = new Put(Bytes.toBytes(cat));
                        put.addColumn(Bytes.toBytes("stats"),
                            Bytes.toBytes("total_amount"),
                            Bytes.toBytes(String.valueOf(total)));
                        put.addColumn(Bytes.toBytes("stats"),
                            Bytes.toBytes("purchase_count"),
                            Bytes.toBytes(String.valueOf(cnt)));
                        put.addColumn(Bytes.toBytes("meta"),
```

```

        Bytes.toBytes("last_update"),
        Bytes.toBytes(String.valueOf(System.currentTimeMillis())));
    table.put(put);
    }
    }
    });
    })
    .trigger(Trigger.ProcessingTime("30 seconds"))
    .start();

    query.awaitTermination();
}
}

```

2.4 Compilation et déploiement

```

# Compiler
mvn clean compile assembly:single

# Copier le JAR dans le conteneur master
docker cp target/ecommerce-pipeline-1-jar-with-dependencies.jar \
    hadoop-master:/root/pipeline.jar

# Copier les libs HBase dans Spark
cp -r $HBASE_HOME/lib/* $SPARK_HOME/jars

# Lancer le job Spark Streaming
spark-submit \
    --class tn.insat.tp5.EcommercePipeline \
    --master local[4] \
    /root/pipeline.jar >> pipeline.log 2>&1 &

```



QUESTION

Pourquoi utilise-t-on foreachBatch plutôt qu'un connecteur HBase natif ? Quels sont les avantages et inconvénients de cette approche ?

2.5 Vérification dans HBase

Après quelques minutes, contrôler que les agrégats sont bien écrits :

```

hbase shell
scan 'sales'
get 'sales', 'Electronique'
exit

```

Résultat attendu :

ROW	COLUMN+CELL
Electronique	column=meta:last_update, value=1710000000000
Electronique	column=stats:purchase_count, value=12

```
Electronique column=stats:total_amount, value=2847.50
Livres       column=stats:purchase_count, value=8
...
```

Partie 3 — Archivage dans HDFS

En parallèle du traitement Spark, archiver tous les événements bruts dans HDFS pour permettre des analyses batch ultérieures.

3.1 Création du répertoire HDFS

```
hdfs dfs -mkdir -p /data/ecommerce/raw
hdfs dfs -ls /data/ecommerce/
```

3.2 Sink HDFS dans Spark

Ajouter le sink HDFS dans EcommercePipeline.java pour écrire tous les événements bruts :

```
// Archivage de TOUS les événements dans HDFS (format JSON)
StreamingQuery hdfsQuery = events.writeStream()
    .outputMode("append")
    .format("json")
    .option("path", "hdfs://localhost:9000/data/ecommerce/raw")
    .option("checkpointLocation", "/tmp/checkpoint-hdfs")
    .trigger(Trigger.ProcessingTime("1 minute"))
    .start();
```

**NOTE**

Le checkpointLocation est obligatoire pour la tolérance aux pannes. Spark l'utilise pour reprendre le traitement depuis le bon offset Kafka en cas de redémarrage.

3.3 Vérification

```
hdfs dfs -ls /data/ecommerce/raw
hdfs dfs -cat /data/ecommerce/raw/part-00000-*.json | head -20
```

**QUESTION**

Vous venez d'implémenter une Lambda Architecture simplifiée. Identifiez la speed layer et la batch layer. Quelles requêtes seriez-vous capables de faire sur chacune ?

Partie 4 — Restitution et discussion

4.1 Présentation par groupe (10 min / groupe)

Chaque groupe présente son pipeline en répondant aux questions suivantes :

- Avez-vous rencontré des problèmes ? Comment les avez-vous résolus ?
- Quels composants ont nécessité le plus de temps de mise en place ?
- Que se passe-t-il si le broker Kafka tombe ? Et si un worker Spark tombe ?

4.2 Questions Tech Lead

?

QUESTION

Notre pipeline écrit dans HBase toutes les 30 secondes. En production avec 1 million d'événements/seconde, quelle modification apporteriez-vous ?

?

QUESTION

HBase a été choisi pour le stockage des agrégats. Auriez-vous pu utiliser Redis, Cassandra ou PostgreSQL ? Quels critères guident ce choix ?

?

QUESTION

Vous devez présenter cette architecture à votre DSI en 5 minutes. Comment justifiez-vous chaque choix technologique en termes de coût, complexité et valeur métier ?

Pour aller plus loin

Pour les groupes qui terminent en avance, ou dans le cadre du projet final :

- Ajouter un tableau de bord de visualisation avec Apache Zeppelin ou Grafana
- Implémenter une détection d'anomalies : alerter si le CA d'une catégorie chute de plus de 50 % en 5 minutes
- Remplacer HBase par Apache Cassandra et comparer les performances d'écriture
- Migrer vers une architecture Kappa : tout traiter via Spark en supprimant la voie HDFS
- Ajouter le chiffrement des données sensibles (montants) avant stockage